2027年3月 卒業見込 ／ ゲームプログラマー職 志望

Mikhail Pavlov

パブロフ ミハイル

ゲームプログラマー — Unity 6 / C# ・ C++ / DirectX 11

- Mirror で**状態変更をすべてサーバー側で処理する**マルチプレイ設計
- 計測を起点にした描画最適化 — **Draw Call 286 → 88（−69%）**
- ドメインロジックを **Unity 非依存アセンブリに分離しユニットテスト**
- C++ / DirectX 11 で**自作エンジンの3Dシューティング**を完成

制作物を見る　▶ プレイ動画　GitHub

PROJECTS

制作物

個人開発 ／ 主力作品

🌾 Make-a-Farm — マルチプレイヤー農場ゲーム

Unity 6 + Mirror。複数プレイヤーが同じ畑を共有する農場ゲームを個人開発しました。

Unity 6000.3　C#　Mirror　Zenject　URP 17.3　NUnit

GitHub →

▶ プレイ動画

🎮 Windows ビルド

担当：設計・実装・最適化のすべて（3Dモデル／サウンドは外部アセット）　**構成：**Mirror の Host/Client 方式







1. クライアントを信用しないゲームプレイ設計

状態を変える処理はすべて `[Command]` でサーバーへ送り、`SyncVar` / `SyncList` の更新はサーバーだけが行う構成に統一。作物の成長はローカル時間ではなく `NetworkTime.time` 基準の期限で管理しています。

→ 途中参加したクライアントでも同じ成長段階が再現され、所持金・アイテムはクライアント側から書き換えられません。

コード： `PlayerShopServer.cs` ／ `Plant.cs`

2. 計測を起点にした描画最適化

Frame Debugger でバッチが分断されている箇所を特定し、①テクスチャをアトラスへ統合、②自作 `MeshCombiner` で静的メッシュを結合、③GPU インスタンシングを適用、の順で対処しました。

286 → 88	**111 → 135**	**9.0 → 7.4 ms**
DRAW CALLS（−69%）	平均 FPS	平均フレームタイム

計測条件：同一シーン・同一設定・同一視点で比較。

コード： `MeshCombiner.cs`

3. ネットワークコードからドメインロジックを切り離す

インベントリのスタック処理を純粋な C# クラス `InventoryModel` へ抽出し、`SyncList` ではなく `IList<InventorySlot>` を受け取る形へ変更。専用 asmdef で Unity 非依存アセンブリに分離しました。

→ Unity を起動せず NUnit で検証可能に（現在はインベントリの7ケース）。

コード： `InventoryModel.cs` ／ `InventoryTest.cs`

4. コードを書かずに作物を追加できるデータ設計

全アイテムは抽象 ScriptableObject `ItemConfig` を基底に `ItemSeed`（種）／`ItemFood`（収穫物）／`ItemCare`（手入れ道具）へ派生。作物を1種類増やす作業は `ItemSeed` アセットを作り、`Stages[]` に成長段階のプレハブ、`TimePerStage` に段階あたりの秒数、`HarvestItem[]` に収穫物、必要なら `CareRequirement`（水やり／施肥／除草／剪定）を差して `Rebuild Database` を押すだけで、C# には触れません。

→ ネットワーク越しに送るのは `Id`（int）だけ。SO 本体はサーバーとクライアントが各自ローカルに持つため、作物を増やしてもネットワークコードは変更不要です。

コード： `ItemSeed.cs` ／ `CareRequirement.cs` ／ `ItemDatabaseBuilder.cs`

🔍 **現時点の範囲：**Host/Client 構成まで。専用サーバー・再接続・セーブ／ロードは未実装で、テスト対象もインベントリのみです。次はセーブ／ロードと `ShopService` の切り出しに取り組みます。

🏆 チーム開発 ／ ゲームジャム

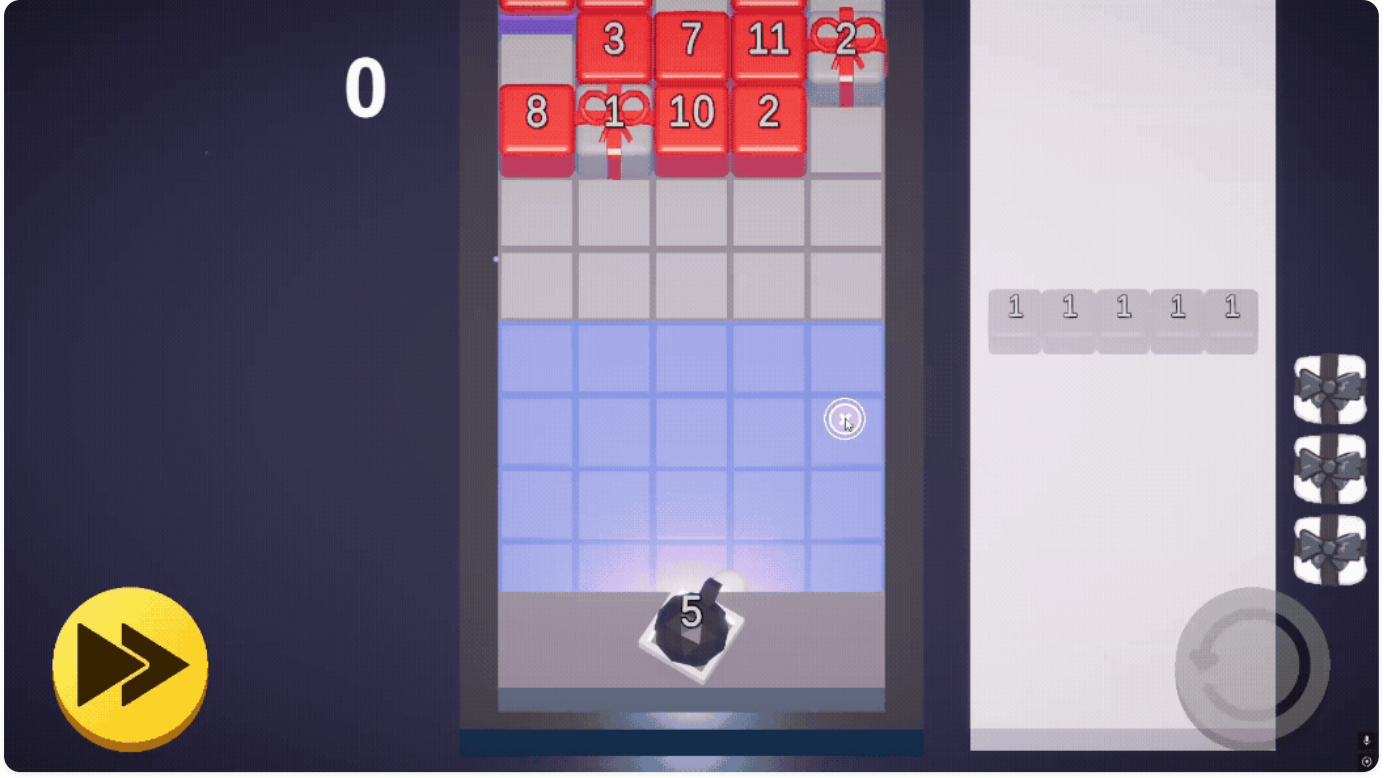
GitHub →

🎮 Tetris Shooter — パズル×シューター

落下するブロックを砲台で撃ち抜いてラインを消す、4名チームでの短期開発。

Unity 6 | C# | URP | VFX | Git / PR運用

担当：Unity 実装・UI・最適化・エフェクト／グラフィック・プロジェクト基盤・Git運用・統合　**チーム：**4名



> **全員が並行して動けるよう、作業境界を設計した**
>
> Unity エディタの経験があるのは自分だけだったため、プロジェクト設定・シーン構成・`.gitignore`・ブランチ運用を自分が用意し、メンバーにはシーンではなくスクリプト単位で実装してもらう分担にしました。上がってきたコードのレビューと統合は自分が担当。
>
> **→ 4名が並行作業してもシーン競合ゼロで完成。空いた時間でエフェクト・UI演出を作り込みました。**

👥 チーム構成（4名）

Mikhail（自分）	Unity 実装・UI・最適化・エフェクト／グラフィック・環境構築／統合
Arai-Eito	プログラミング
akitakann	企画・プランニング
Mizuno	サウンド／音楽

学び：分担の設計そのものが技術課題でした。統合を最後にまとめると破綻するため、以降は早い段階から統合を回すようにしています。

個人開発 ／ 自作エンジン

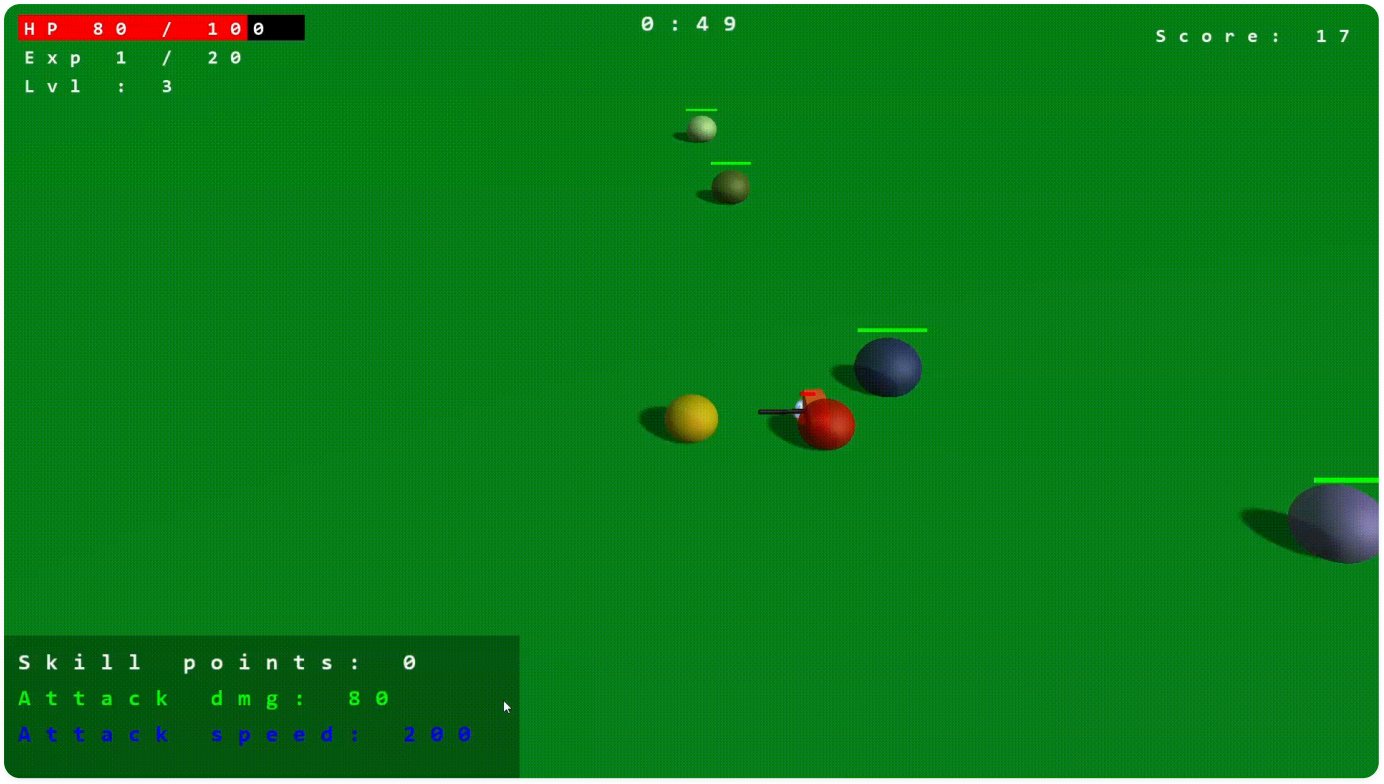
GitHub →

👾 Slime Invasion 3D — C++ / DirectX 11 製 3D シューティング

エンジンを使わず、Unity が内部で行っている処理を自分の手で再現することが目的でした。

C++ DirectX 11 HLSL Assimp DirectXTex

担当：設計・実装すべて（個人開発）　**期間：**約6ヶ月



描画パイプラインとスキニングを自前で組む

デバイス／スワップチェーンの初期化、各種バッファの生成と更新、入力レイアウト、深度ステンシル、シェーダーのコンパイルとバインドまでを実装。アニメーションは Assimp でボーン階層とウェイトを読み込み、ボーン行列を定数バッファへ送って頂点シェーダー側でスキニングしています。ほかに Phong ライティング（HLSL）、AABB / 球体の当たり判定、ビルボードのパーティクル、ウェーブ制の敵AIを実装。

→ Unity の `Material` / `SetPass` / `SkinnedMeshRenderer` が内部で何をしているかを説明でき、最適化を判断する根拠になっています。

SKILLS

技術スキル

Core
設計判断まで自分で行える

C# | Unity 6 / URP | Mirror（サーバー権威型設計） | 描画最適化・プロファイリング | ドメインロジック分離・ユニットテスト | ScriptableObject データ設計

Additional
作品の中で実装した

C++ / DirectX 11 / HLSL | Zenject（DI） | New Input System | Cinemachine | uGUI / TextMesh Pro | エディタ拡張 | Git / プルリクエスト運用

Familiar
基礎を習得済み

Unity ECS / DOTS | SQL / SQLite | Assimp | Blender

次に取り組むこと：GitHub Actions（GameCI）でのテスト自動実行 ／ セーブ・ロードと再接続時の状態復元 ／ `ShopService` の切り出しとテスト拡張 ／ Unity ECS の適用検討。

GET IN TOUCH

連絡先

HAL東京 ゲームプログラム学科（2025年4月入学・2027年3月 卒業見込）／ 2027年4月 就業開始可能 ／ 在留資格：留学 ／ 日本語：ビジネスレベル

新卒採用のほか、在学中のインターンシップも歓迎です。コードについてのご質問にはいつでもお答えします。

✉️ pavlovmikhailn1@gmail.com

🐙 github.com/mikhailroug

© 2026 Mikhail Pavlov — Game Programmer Portfolio（Unity / C# / C++ / DirectX 11）